

Change the line in `main()` so it instantiates a `MiniBoard` object instead of a `TTTBoard` object:

```
if input('Use mini board? Y/N: ').lower().startswith('y'):
    gameBoard = MiniBoard() # Create a MiniBoard object.
else:
    gameBoard = TTTBoard() # Create a TTTBoard object.
```

Other than this one line change to `main()`, the rest of the program works the same as before. When you run the program now, the output will look something like this:

```
Welcome to Tic-Tac-Toe!
Use mini board? Y/N: y

... 123
... 456
... 789
What is X's move? (1-9)
1

X.. 123
... 456
... 789
What is O's move? (1-9)
--snip--
XXX 123
.OO 456
O.X 789
X has won the game!
Thanks for playing!
```

Your program can now easily have both implementations of these tic-tac-toe board classes. Of course, if you *only* want the mini version of the board, you could simply replace the code in the `getBoardStr()` method for `TTTBoard`. But if you need *both*, inheritance lets you easily create two classes by reusing their common code.

If we didn't use inheritance, we could have, say, added a new attribute to `TTTBoard` called `useMiniBoard` and put an `if-else` statement inside `getBoardStr()` to decide when to show the regular board or the mini one. This would work well for such a simple change. But what if the `MiniBoard` subclass needed to